TU WIEN Informatics

Titel der Arbeit

Optionaler Untertitel der Arbeit

BACHELORARBEIT

zur Erlangung des akademischen Grades

Bachelor of Science

im Rahmen des Studiums

Informatik mit Vertiefung Digital Health

eingereicht von

Sebastian Bolek
Matrikelnummer 12323843

an der Fakultät für Informatik

der Technischen Universität Wien

Betreuung: Associate Prof. Dr.in Renata Georgia Raidou
Mitwirkung: Pretitle Forename Surname, Posttitle
Pretitle Forename Surname, Posttitle
Pretitle Forename Surname, Posttitle

Wien, 1. Jänner 2001

Sebastian Bolek	Renata Georgia Raidou

TU WIEN Informatics

Title of the Thesis

Optional Subtitle of the Thesis

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Informatics with specialization in Digital Health

by

Sebastian Bolek
Registration Number 12323843

to the Faculty of Informatics

at the TU Wien

Advisor: Associate Prof. Dr.in Renata Georgia Raidou
Assistance: Pretitle Forename Surname, Posttitle
Pretitle Forename Surname, Posttitle
Pretitle Forename Surname, Posttitle

Vienna, January 1, 2001

Sebastian Bolek	Renata Georgia Raidou

Erklärung zur Verfassung der Arbeit

Sebastian Bolek

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.
Ich erkläre weiters, dass ich mich generativer KI-Tools lediglich als Hilfsmittel bedient habe und in der vorliegenden Arbeit mein gestalterischer Einfluss überwiegt. Im Anhang „Übersicht verwendeter Hilfsmittel“ habe ich alle generativen KI-Tools gelistet, die verwendet wurden, und angegeben, wo und wie sie verwendet wurden. Für Textpassagen, die ohne substantielle Änderungen übernommen wurden, habe ich jeweils die von mir formulierten Eingaben (Prompts) und die verwendete IT-Anwendung mit ihrem Produktnamen und Versionsnummer/Datum angegeben.

Wien, 1. Jänner 2001

Sebastian Bolek

Danksagung

Ihr Text hier.

Acknowledgements

Enter your text here.

Kurzfassung

Ihr Text hier.

Abstract

Enter your text here.

Contents

CHAPTER 1

Introduction

Enter your text here.

CHAPTER 2

Additional Chapter

Enter your text here.

CHAPTER 3

Methodology

3.1 Overview of the approach

The goal of this work is a solid, thickness-accurate 3D model of the valve leaflets that can serve as the basis for an ultrasound phantom. The starting point is two things: a volumetric ultrasound scan of the valve, stored as a VDB volume, and a surface mesh of the leaflets. The problem is that these two do not directly provide what is needed. The scan contains the full valve, but only as a grid of grayscale values with no explicit geometry. The mesh has clean geometry, but it is only a single surface with no thickness, so on its own it cannot be turned into a physical object with a wall.

The core task of the whole pipeline is therefore to bridge these two: to bring the mesh and the volume into a common coordinate system, to recover the leaflet thickness from the volume, to transfer that thickness to the mesh, and to turn the single surface into a closed solid.

I did not carry out all of this in a single program, because no single tool does every part well. Instead the work is split across three: MeVisLab is used to load and prepare the volume, 3D Slicer is used to align the mesh with the volume and to take the thickness measurements, and Blender is used to interpolate the measurements over the mesh and build the solid. Some smaller steps, mainly the interpolation, were done with short Python scripts using the numpy librariy. The following sections describe each part, and, more importantly, why each choice was made and not one of the other possible ones.

3.2 Input data

The first input is the volumetric scan. It is a 3D grayscale field where bright voxels roughly correspond to tissue and dark voxels to the surrounding space. It contains the leaflets with their real thickness, but this thickness is only present as a bright band in

the image and not as a number or a shape that can be measured automatically without further steps. The scan is also not clean everywhere: the ultrasound probe used to record it left a very noisy region in the centre of the valve, around the opening, which becomes important later.

The second input is the surface mesh of the leaflets. This mesh is a single surface. It does not enclose a volume, and it has no wall thickness at all. In practice there are two separate leaflet meshes, referred to here as [Mesh1] and [Mesh2].

Before deciding how to add thickness, I first had to establish what the surface actually represents, because this changes the whole calculation. A surface mesh of a thin structure can either be a *midsurface*, sitting in the middle between the two walls, or a *boundary surface*, sitting on one side. If it is a midsurface, the real thickness is split evenly on both sides of it. If it is a boundary surface, the whole thickness sits on one side only. Getting this wrong would make every thickness value off by a factor of two.

To check this, the mesh was placed next to the volume and a cross-section through a leaflet was examined. If the mesh were a midsurface, it would run through the middle of the bright leaflet band. In this case the surface sits on one side of the band, which means it is the underside (a boundary surface), not the middle. This has a direct consequence for the later steps: the full measured thickness has to be added on one side, and there is no division by two. I come back to this in Section 3.7.

3.3 Choosing how to recover thickness

Thickness is not stored anywhere in the data. It has to be recovered from the grayscale volume. There were two reasonable ways to do this, and it is worth explaining both, because the reason for choosing the second one is part of the method.

The first idea was to compute the thickness fully automatically with a distance transform. The steps would be: turn the volume into a binary mask, where material is 1 and everything else is 0, and then run a distance transform on that mask. A distance transform gives, for every inside voxel, the distance to the nearest background voxel, in other words the distance to the closest wall [edM]. In the middle of a leaflet, that distance is half the local thickness, so the thickness could in principle be read off as two times the distance value. This was set up in MeVisLab using an `OpenVDBLoad` module to load the volume, a `Threshold` module to make the binary mask (values above 0.4 set to 1, everything else to 0), and a `EuclideanDistanceTransform` module in 3D mode to produce the distance field.

The reason this idea is attractive is that it is automatic and dense: it would give a thickness value everywhere on the leaflet at once, without any manual work. In the end, though, I decided not to use it, for a few reasons that add up.

First, there is a practical problem with reading the result. To get thickness at the mesh, the distance field has to be sampled exactly at the positions of the mesh points. MeVisLab

does not have a simple, ready-made module for "sample this volume at the vertices of that mesh", and the scripting module has no image input or output that would let this be done directly inside the network. It would mean exporting the distance field and doing the sampling externally in Python, which is possible but adds another step that would itself have to be checked.

Second, the simple "two times the distance" rule only holds if the mesh is a midsurface. As shown in Section 3.2, the mesh here is the underside. For a boundary surface the correct value would be the distance from the underside across to the top wall, which is not what a symmetric distance transform gives without extra work. So the clean version of the method does not actually match this data.

Third, and most importantly, the whole result depends on one global threshold that separates material from background, and the volume is not clean enough for that. The Ultrasound volume has speckled and blurred edges, and this scan has an especially noisy region in the centre of the valve. A single fixed threshold applied to the whole volume would turn that noise into false geometry exactly where the data is worst, and there would be no easy way to notice that it had happened. A dense automatic field of numbers is only useful if it can be trusted, and here it could not be.

Taken together, these points describe a method that would produce a lot of numbers that could not be properly checked. For a bachelor's thesis, a dense automatic result that cannot be verified is worse than a smaller set of values that can. This is the reason I chose to measure the thickness by hand at a limited number of well-imaged locations instead. Measuring by hand, however, only works if the mesh is correctly placed inside the volume, so the first step before measuring was to align the two. This is described next.

3.4 Aligning the mesh and the volume

The mesh and the volume did not overlap when they were first loaded together in both MeVisLab and 3D Slicer - only Blender could directly open the .usda file which preserved both coordinate systems in the correct orientation; in MeVisLab and 3D Slicer they sat in different coordinate systems. This alignment matters here because, as described in Section 3.5, the thickness is measured along lines perpendicular to the mesh surface. If the mesh is in the wrong place, those perpendicular lines start in the wrong place, and the measurement is meaningless. So the mesh and the volume had to be brought into the same coordinate frame first.

Aligning the two by their shapes directly is difficult, because the leaflet surface is smooth and curved and offers no clear matching landmarks to line up by eye. Instead, I used a reference that all three programs agree on: the bounding box. MeVisLab, 3D Slicer and Blender each draw an axis-aligned bounding box around the volume, and this box is the same in all of them because it is derived from the same volume. Since everything was perfectly alligned in Blender I added planes to the mesh that correspond to the

bounding box of the volume, so that the mesh carries the same box as a built-in reference. Matching the mesh's bounding planes to the volume's bounding box then defines the alignment completely, because the two boxes have to coincide.

From this box-to-box correspondence a transformation matrix was obtained, which was applied to the mesh in 3D Slicer. After applying it, the mesh sits inside the volume in the correct position and orientation, so that a leaflet in the mesh lines up with the corresponding bright band in the scan. Using the bounding box as a shared reference is more reliable here than trying to register the curved surfaces against each other, and it keeps the alignment reproducible, because the box is defined by the data and not by a subjective judgement of where the surfaces should meet.

3.5 Measuring leaflet thickness in 3D Slicer

With the mesh aligned, the thickness was measured in 3D Slicer [FBKC+12]. The tool used is the line (ruler) markup, which gives a direct distance reading in millimetres between two points. The important part is not the tool but the rule I followed for placing the two points, because a clear rule is what makes the measurements repeatable.

I chose not to judge the leaflet edges purely by eye, and I also chose not to threshold the whole volume, for the reasons given in Section 3.3. Instead, I used a local intensity criterion, applied only at the spot being measured. Tissue was defined as an intensity of 0.2 or higher. Each measurement line was placed perpendicular to the mesh surface at the chosen location. Along that line, the measurement starts where the intensity first rises to 0.2 and ends where it falls back on or below 0.2, and the thickness is the length of that span. This is the same kind of intensity criterion that the automatic approach would have used, but applying it locally, along a single line at a spot I had judged to be clean, avoids the problem of applying it to the whole noisy volume at once. I chose where to place each line; the intensity rule then decided where the tissue began and ended.

Where the measurements were taken was itself a choice driven by the data quality. Thickness is not the same everywhere on a leaflet, so I wanted to sample across the surface, but two regions turned out to be hard to measure. The first is the centre of the valve, around the opening, where the probe left a noisy hole in the scan. In that region it is difficult to see a clean tissue band, so reliable perpendicular measurements are hard to obtain. The second is the thin outer brim of the leaflet, at the edge of the mesh, where measurements were also difficult to take cleanly. In between these two problem regions the data is clearer, so most of the measurements were taken there, where the tissue band is well defined and the intensity rule can be applied with confidence.

For the two difficult regions I made a deliberate simplification rather than forcing measurements the data does not support. For the noisy centre I took all the measurements I could obtain there and reduced them to a single value, the median, and treated the thickness across the whole centre region as constant at that value. I did the same for the outer edge, taking the median of the edge measurements and treating the edge as

having one constant thickness. The median was used rather than the mean because it is less affected by the occasional bad reading, which is exactly what one expects in these noisy regions. Assuming a homogeneous thickness in the centre and at the edge is a simplification, and it is stated as such, but it is a reasonable one: the data in those regions is not good enough to justify anything finer, and a single well-chosen value is more honest than a set of unreliable point measurements.

3.6 Spreading the measurements over the mesh

At this point there is, for each leaflet, a set of thickness values: several point measurements in the well-imaged middle region, plus one constant value for the noisy centre and one constant value for the outer edge. The mesh, however, has several thousand vertices, and the step that builds the solid (Section 3.7) needs a thickness value at every single vertex. So the measured values have to be spread over the whole surface.

The measured points can be recorded as follows:

Point	Location	Leaflet	Thickness [mm]
1	Base	[Mesh1]	[]
2	Middle	[Mesh1]	[]
3	Free edge (median)	[Mesh1]	[]
4	Centre (median)	[Mesh1]	[]
...	...	...	...

Table 3.1: Measured leaflet thickness at the sampled locations.

The simplest way to fill the gap would be to give the whole leaflet one average thickness. I rejected this because it would remove the variation between the base and the edge that I went to the trouble of capturing. So some form of interpolation is needed for the region that is well sampled, while the centre and the edge keep their constant median values as described in Section 3.5.

For the interpolation I chose inverse distance weighting (IDW) [She68]. The idea is easy to state: each vertex is given a thickness that is a weighted average of the known values, and values that are closer to the vertex count more than ones further away. The weight of each value falls off with distance, following one over the distance raised to a power. A vertex sitting right next to a known value gets almost exactly that value, and a vertex halfway between two of them gets roughly their average.

I chose IDW over more advanced interpolation methods such as kriging or radial basis functions on purpose. With only a handful of known values, the more advanced methods bring in extra assumptions and parameters that could not be checked with so little data. IDW, in contrast, makes no assumption about an underlying function, works with irregularly placed values, and is simple enough to explain and defend in full. For

the power parameter, a value of two was used. This gives greater weight to nearby measurements while still providing a smooth transition between the measured points.

The interpolation was carried out in Blender using a short Python script, as shown in Listing 3.1. NumPy [HMVDW+20] is used for the distance and weighted averaging calculations. For each vertex, the script calculates the Euclidean distance to all measured points, determines the corresponding inverse-distance weights, and computes the weighted average of the measured thickness values. For every vertex the script computes the distances to the known values, forms the weights, and takes the weighted average. The result is stored on the mesh as a per-vertex weight, called a vertex group in Blender. Before storing, the values are scaled to the range zero to one by dividing by the largest thickness, because the modifier used in the next step expects weights in that range.

```
import bpy
import numpy as np


def apply_thickness(obj_name, known_pos, known_val, power=2):
    obj = bpy.data.objects[obj_name]
    mesh = obj.data

    all_verts = np.array([v.co for v in mesh.vertices])

    def idw(query, kpos, kval, power):
        diff = query[:, None, :] - kpos[None, :, :]
        dist = np.maximum(np.linalg.norm(diff, axis=2), 1e-9)

        w = 1.0 / dist**power

        return (w * kval).sum(1) / w.sum(1)

    thickness_mm = idw(
        all_verts,
        np.array(known_pos),
        np.array(known_val),
        power
    )

    max_d = np.array(known_val).max()

    # Remove existing group for re-runs
    if "thickness" in obj.vertex_groups:
        obj.vertex_groups.remove(
            obj.vertex_groups["thickness"]
        )

    vg = obj.vertex_groups.new(name="thickness")

    for i, d in enumerate(thickness_mm):
```

```
        vg.add(
            [i],
            float(d / max_d),
            'REPLACE'
        )

    print(
        f"{obj_name}: "
        f"{thickness_mm.min():.2f} - "
        f"{thickness_mm.max():.2f} mm "
        f"(max={max_d})"
    )

# ------------------------------------------------------------------
# Mesh 1
# ------------------------------------------------------------------
apply_thickness(
    "mesh1",
    known_pos=[
        [point1x,  point1y,  point1z],
        [point2x,  point2y,  point2z],
        [point3x,  point3y,  point3z],
        [point4x,  point4y,  point4z],
        [point5x,  point5y,  point5z],
        [point6x,  point6y,  point6z],
        [point7x,  point7y,  point7z],
        [point8x,  point8y,  point8z],
        [point9x,  point9y,  point9z],
        [point10x, point10y, point10z],
        [point11x, point11y, point11z],
    ],
    known_val=[
        thickness1,
        thickness2,
        thickness3,
        thickness4,
        thickness5,
        thickness6,
        thickness7,
        thickness8,
        thickness9,
        thickness10,
        thickness11,
    ]
)

# ------------------------------------------------------------------
# Mesh 2
# ------------------------------------------------------------------
```

```
apply_thickness(
    "mesh2",
    known_pos=[
        [point1x, point1y, point1z],
        [point2x, point2y, point2z],
        [point3x, point3y, point3z],
        [point4x, point4y, point4z],
        [point5x, point5y, point5z],
        [point6x, point6y, point6z],
    ],
    known_val=[
        thickness1,
        thickness2,
        thickness3,
        thickness4,
        thickness5,
        thickness6,
    ]
)
```

Listing 3.1: Interpolation of the thickness distribution in Blender using inverse distance weighting.

One limitation of this choice should be stated openly, because it also explains a later decision. IDW here uses straight-line (euclidean) distance through space, not distance measured along the curved surface. Across a fold, or across the gap between the two leaflets, straight-line distance could mix values that really should not be mixed, because two points can be close in space while being far apart along the tissue. This is exactly why the two leaflets are kept as two separate meshes and interpolated separately: a value measured on one leaflet cannot leak across the gap onto the other. Within a single, fairly flat leaflet the difference between straight-line distance and along-surface distance is small enough to accept for the purpose of this work.

3.7 Building the solid model

The last step has this problem to solve: there is now a thickness value at every vertex, and a single surface that represents the underside of the leaflet, and this has to be turned into a closed solid whose wall has the correct local thickness.

For this step, Blender's Solidify modifier was used [Ble26]. This modifier takes a surface and extrudes it by a chosen thickness to produce a solid, and, importantly, it can scale that thickness per vertex using a vertex group. This is what makes it fit the situation here: the interpolated thickness field from Section 3.6 is stored as exactly such a vertex group, so it maps directly onto how far each part of the surface is pushed out. In practice the modifier's thickness is set to the largest thickness value, given in metres because Blender works in metres, and the zero-to-one weights then scale every other part of the

surface below that maximum. The local thickness at a vertex is the weight at that vertex multiplied by the modifier thickness.

The extrusion is done in one direction only, which is where the check from Section 3.2 finally pays off. Because the mesh is the underside of the leaflet and not the midsurface, the whole thickness has to be added on one side, growing outward from the surface, rather than split half on each side. This is also the concrete reason why the "two times the distance" rule from the distance-field idea in Section 3.3 would have been wrong for this data: a factor of two is only correct when the surface is the centre, and here it is a boundary. In the modifier this is set by using a one-sided offset so that the material grows only outward. The direction of the extrusion follows the surface normals, so the normals were checked beforehand and flipped where needed, to make sure the wall grows away from the leaflet body and not into it.

Each leaflet is solidified on its own before the two are combined. The reason is that the two leaflets have different thickness ranges, and a single modifier can only hold one thickness value and one vertex group. So [Mesh1] and [Mesh2] are each given their own modifier with their own maximum thickness. Only after the thickness has been fixed into the geometry, by applying the modifier, are the two meshes joined into one object. Joining them earlier would force a single shared thickness on both, and would also let the interpolation bleed across the gap between them, for the reason given in Section 3.6.

The output of this step is a single solid mesh in which the leaflet walls have the measured, locally varying thickness.

3.8 Printing the complete model

Once a sufficiently accurate approximation, or model, of the mitral valve had been successfully developed, the actual objective and core of this work was addressed: the fabrication of a realistic ultrasound phantom. As accuracy and realism were prioritized over ease of production and scalability, FDM 3D printing was selected as the manufacturing method [MCZS19]. Modern 3D printers provide sufficient detail resolution and, importantly for this project, the ability to work with flexible materials such as TPU. The fabrication of flexible structures using molds and a material such as silicone would also have been possible and could even have enabled scalable production. However, 3D printing was ultimately chosen because it allows for easy modifications to the model and eliminates the need to consider how the mold can be removed after casting.

Throughout the project, while the thickness measurements were being determined, test prints were conducted in parallel to identify the approach that would yield the best possible result. It was apparent that the heart valve would require support structures during printing, as the convex geometry would need to rest as flat as possible on the build plate, resulting in overhangs of well above 60° [MCZS19]. Printing the model vertically slightly reduced the cross-sectional area requiring support, but not to a significant extent. Furthermore, this orientation caused the structure to become unstable during printing, as

the build plate moves back and forth along the Y-axis. The higher the print progressed, the more pronounced this movement became, causing the structure to wobble.

A major disadvantage of support material is the difficulty of completely removing all residues and achieving a smooth surface after printing. Test prints using PLA, for example, which is a relatively rigid plastic, demonstrated that removing the support structures alone was already a considerable challenge. Achieving a surface as smooth as the upper side of the print would then require substantial additional effort. Printing with flexible TPU did not significantly change this situation. The TPA95A used was specified with a Shore A hardness of 95; in general, lower Shore A values correspond to softer and more flexible TPU materials [reca]. It could be printed using settings that were almost identical to those used for PLA. For TPU82A [recc], the printing speed was reduced to avoid compromising print quality. However, at the desired thickness of the heart valve, this material remained too inflexible. TPU60A [recb], which provided an appropriate degree of flexibility and softness, introduced another problem: the amount of support material used previously was insufficient to adequately support the material between the individual support structures. The resulting print contained numerous holes and areas where the printed material had fused with the supports. The solution was to increase the density of the support structures by reducing the spacing between them. In this case, this was achieved by selecting smaller squares in the grid pattern.

This resulted in a print using the desired material that appeared intact at first glance. However, the issue of the difficult-to-remove support structures remained. To address this problem, additional prototypes were produced using a multi-nozzle printer capable of printing with multiple materials simultaneously. The idea was to print the supports using a water-soluble filament, in this case polyvinyl alcohol (PVA) [MKM+23], while continuing to print the mitral valve using TPU60A. The finished model could then have been placed in water, allowing the supports to dissolve completely without leaving any residue.

However, there were concerns that the TPU might not adhere sufficiently to the PVA supports during printing, potentially resulting in print defects in the model. Therefore, PLA supports were initially used instead. Ultimately, this proved to be the right choice for this particular application, as the PLA and TPU adhered strongly enough to ensure a successful print while still allowing the valve to be easily peeled away from the support material after completion. As hoped, this resulted in a perfectly smooth underside. Furthermore, the gap within the valve opening was preserved.

The only remaining step was to manually create a small incision in the opening using a scalpel. This was necessary because 3D printing small openings generally leaves behind thin strands of filament, a phenomenon known as *stringing*, which can cause the opening to become partially sealed. The completed prototype, including the support structures, was handed over to Siemens Healthineers, where the valve is currently undergoing further testing.

change if Siemens gets back with results

CHAPTER 4

Introduction to LaTeX

Since LaTeX is widely used in academia and industry, there exists a plethora of freely accessible introductions to the language. Reading through the guide at `https://en.wikibooks.org/wiki/LaTeX` serves as a comprehensive overview of most of the functionality and is highly recommended before starting with a thesis in LaTeX.

4.1 Installation

A full LaTeX distribution consists not only of the binaries that convert the source files to the typeset documents but also of a wide range of packages and their documentation. Depending on the operating system, different implementations are available as shown in Table 4.1. **Due to the large number of packages that are in everyday use and due to their high interdependence, it is paramount to keep the installed distribution up to date.** Otherwise, obscure errors and tedious debugging ensue.

4.2 Editors

A multitude of TeX editors are available differing in their editing models, their supported operating systems, and their feature sets. A comprehensive overview of editors can be

Distribution	Unix	Windows	MacOS
TeX Live	**yes**	yes	(yes)
MacTeX	no	no	**yes**
MikTeX	(yes)	**yes**	yes

Table 4.1: TeX/LaTeX distributions for different operating systems. Recommended choice is in **bold**.

	Description
1	Scan for refs, toc/lof/lot/loa items and cites
2	Build the bibliography
3	Link refs and build the toc/lof/lot/loa
4	Link the bibliography
5	Build the glossary
6	Build the acronyms
7	Build the index
8	Link the glossary, acronyms, and the index
9	Link the bookmarks

	Command
1	`pdflatex.exe  example`
2	`bibtex.exe  example`
3	`pdflatex.exe  example`
4	`pdflatex.exe  example`
5	`makeindex.exe -t example.glg -s example.ist -o example.gls example.glo`
6	`makeindex.exe -t example.alg -s example.ist -o example.acr example.acn`
7	`makeindex.exe -t example.ilg -o example.ind example.idx`
8	`pdflatex.exe  example`
9	`pdflatex.exe  example`

Table 4.2: Compilation steps for this document. The following abbreviations were used: table of contents (toc), list of figures (lof), list of tables (lot), list of algorithms (loa).

found on the Wikipedia page `https://en.wikipedia.org/wiki/Comparison_of_TeX_editors`. TeXstudio (`http://texstudio.sourceforge.net/`) is recommended. Most editors support synchronization of the generated document and the LaTeX source by `Ctrl`-clicking either on the source document or the generated document.

4.3 Compilation

Modern editors usually provide the compilation programs to generate Portable Document Format (PDF) documents and for most LaTeX source files, this is sufficient. More advanced LaTeX functionality, such as glossaries and bibliographies, needs additional compilation steps, however. It is also possible that errors in the compilation process invalidate intermediate files and force subsequent compilation runs to fail. It is advisable to delete intermediate files (`.aux`, `.bbl`, etc.), if errors occur and persist. All files that are not generated by the user are automatically regenerated. To compile the current document, the steps as shown in Table 4.2 have to be taken.

4.4 Basic Functionality

In this section, various examples are given of the fundamental building blocks used in a thesis. Many LaTeX commands have a rich set of options that can be supplied as optional arguments. The documentation of each command should be consulted to get an impression of the full spectrum of its functionality.

4.4.1 Floats

Two main categories of page elements can be differentiated in the usual LaTeX workflow: *(i)* the main stream of text and *(ii)* floating containers that are positioned at convenient positions throughout the document. In most cases, tables, plots, and images are put into such containers since they are usually positioned at the top or bottom of pages. These are realized by the two environments `figure` and `table`, which also provide functionality for cross-referencing (see Table 4.3 and Figure 4.1) and the generation of corresponding entries in the list of figures and the list of tables. Note that these environments solely act as containers and can be assigned arbitrary content.

4.4.2 Tables

A table in LaTeX is created by using a `tabular` environment or any of its extensions, e.g., `tabularx`. The commands `\multirow` and `\multicolumn` allow table elements to span multiple rows and columns.

Position		
Group	Abbrev	Name
Goalkeeper	GK	Paul Robinson
Defenders	LB	Lucus Radebe
	DC	Michael Duburry
	DC	Dominic Matteo
	RB	Didier Domi
Midfielders	MC	David Batty
	MC	Eirik Bakke
	MC	Jody Morris
Forward	FW	Jamie McMaster
Strikers	ST	Alan Smith
	ST	Mark Viduka

Table 4.3: Adapted example from the LaTeX guide at `https://en.wikibooks.org/wiki/LaTeX/Tables`. This example uses rules specific to the `booktabs` package and employs the multi-row functionality of the `multirow` package.

4.4.3 Images

An image is added to a document via the `\includegraphics` command as shown in Figure 4.1. The `\subcaption` command can be used to reference subfigures, such as Figure 4.1a and 4.1b.

(a) The TU Wien Informatics logo at text width.

(b) The TU Wien Informatics logo at half the text width.

Figure 4.1: The header logo at different sizes.

4.4.4 Mathematical Expressions

One of the original motivations for creating the TeX system was the need for mathematical typesetting. To this day, LaTeX is the preferred system to write math-heavy documents and a wide variety of functions aids the author in this task. A mathematical expression can be inserted inline as $\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$ outside of the text stream as

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

or as a numbered equation with

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}. \tag{4.1}$$

4.4.5 Pseudo Code

The presentation of algorithms can be achieved with various packages; the most popular are `algorithmic`, `algorithm2e`, `algorithmicx`, or `algpseudocode`. An overview is given at `https://tex.stackexchange.com/questions/229355`. An example of the use of the `alogrithm2e` package is given with Algorithm 4.1.

Algorithm 4.1: Gauss-Seidel

Input: A scalar ϵ, a matrix $\mathbf{A} = (a_{ij})$, a vector $\vec{b}$, and an initial vector $\vec{x}^{(0)}$
Output: $\vec{x}^{(n)}$ with $\mathbf{A}\vec{x}^{(n)} \approx \vec{b}$

for $k \leftarrow 1$ **to** *maximum iterations* **do**
  for $i \leftarrow 1$ **to** n **do**
    $x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k)} - \sum_{j>i} a_{ij} x_j^{(k-1)} \right)$;
  end
  if $|\vec{x}^{(k)} - \vec{x}^{(k-1)}| < \epsilon$ **then**
    break for;
  end
end
return $\vec{x}^{(k)}$;

4.5 Bibliography

The referencing of prior work is a fundamental requirement of academic writing and is well supported by LaTeX. The BibTeX reference management software is the most commonly used system for this purpose. Using the `\cite` command, it is possible to reference entries in a `.bib` file out of the text stream, e.g., as [**?**]. The generation of the formatted bibliography needs a separate execution of `bibtex.exe` (see Table 4.2).

4.6 Table of Contents

The table of contents is automatically built by successive runs of the compilation, e.g., of `pdflatex.exe`. The command `\setsecnumdepth` allows the specification of the depth of the table of contents and additional entries can be added to the table of contents using `\addcontentsline`. The starred versions of the sectioning commands, i.e., `\chapter*`, `\section*`, etc., remove the corresponding entry from the table of contents.

4.7 Acronyms / Glossary / Index

The list of acronyms, the glossary, and the index need to be built with a separate execution of `makeindex` (see Table 4.2). Acronyms have to be specified with `\newacronym` while glossary entries use `\newglossaryentry`. Both are then used in the document content with one of the variants of `\gls`, such as `\Gls`, `\glspl`, or `\Glspl`. Index items are simply generated by placing `\index{`⟨*entry*⟩`}` next to all the words that correspond to the index entry ⟨*entry*⟩. Note that many enhancements exist for these functionalities and the documentation of the `makeindex` and the `glossaries` packages should be consulted.

4.8 Tips

Since TeX and its successors do not employ a What You See Is What You Get (WYSIWYG) editing scheme, several guidelines improve the readability of the source content:

- Each sentence in the source text should start with a new line. This helps not only the user navigate through the text but also enables revision control systems (e.g. Subversion (SVN), Git) to show the exact changes authored by different users. Paragraphs are separated by one (or more) empty lines.
- Environments, which are defined by a matching pair of `\begin{name}` and `\end{name}`, can be indented by whitespace to show their hierarchical structure.
- In most cases, the explicit use of whitespace (e.g. by adding `\hspace{4em}` or `\vspace{1.5cm}`) violates typographic guidelines and rules. Explicit formatting should only be employed as a last resort and, most likely, better ways to achieve the desired layout can be found by a quick web search.
- The use of bold or italic text is generally not supported by typographic considerations and the semantically meaningful `\emph{...}` should be used.

The predominant application of the LaTeX system is the generation of PDF files via the PdfLaTeX binaries. In the current version of PdfLaTeX, it is possible that absolute file paths and user account names are embedded in the final PDF document. While this poses only a minor security issue for all documents, it is highly problematic for double-blind reviews. The process shown in Table 4.4 can be employed to strip all private information from the final PDF document.

	Command
1	Rename the PDF document `final.pdf` to `final.ps`.

2	Execute the following command:
	`ps2pdf -dPDFSETTINGS#/prepress ^`
	`-dCompatibilityLevel#1.4 ^`
	`-dAutoFilterColorImages#false ^`
	`-dAutoFilterGrayImages#false ^`
	`-dColorImageFilter#/FlateEncode ^`
	`-dGrayImageFilter#/FlateEncode ^`
	`-dMonoImageFilter#/FlateEncode ^`
	`-dDownsampleColorImages#false ^`
	`-dDownsampleGrayImages#false ^`
	`final.ps final.pdf`

On Unix-based systems, replace # with = and ^ with \.

Table 4.4: Anonymization of PDF documents.

4.9 Resources

4.9.1 Useful Links

In the following, a listing of useful web resources is given.

`https://en.wikibooks.org/wiki/LaTeX` An extensive wiki-based guide to LaTeX.

`http://www.tex.ac.uk/faq` A (huge) set of Frequently Asked Questions (FAQ) about TeX and LaTeX.

`https://tex.stackexchange.com/` The definitive user forum for non-trivial LaTeX-related questions and answers.

4.9.2 Comprehensive TeX Archive Network (CTAN)

The CTAN is the official repository for all TeX-related material. It can be accessed via `https://www.ctan.org/` and hosts (among other things) a huge variety of packages that provide extended functionality for TeX and its successors. Note that most packages contain PDF documentation that can be directly accessed via CTAN.

In the following, a short, non-exhaustive list of relevant CTAN-hosted packages are given together with their relative path.

algorithm2e Functionality for writing pseudo code.

amsmath Enhanced functionality for typesetting mathematical expressions.

amssymb Provides a multitude of mathematical symbols.

booktabs Improved typesetting of tables.

enumitem Control over the layout of lists (`itemize`, `enumerate`, `description`).

fontenc Determines font encoding of the output.

glossaries Create glossaries and lists of acronyms.

graphicx Insert images into the document.

inputenc Determines encoding of the input.

l2tabu A description of bad practices when using LaTeX.

mathtools Further extension of mathematical typesetting.

memoir The document class upon which the `vutinfth` document class is based.

multirow Allows table elements to span several rows.

pgfplots Function plot drawings.

pgf/TikZ Creating graphics inside LaTeX documents.

subcaption Allows the use of subfigures and enables their referencing.

symbols/comprehensive A listing of around 5000 symbols that can be used with LaTeX.

voss-mathmode A comprehensive overview of typesetting mathematics in LaTeX.

xcolor Allows the definition and use of colors.

Overview of Generative AI Tools Used

Ihr Text hier.

Übersicht verwendeter Hilfsmittel

Enter your text here.

List of Figures

List of Tables

List of Algorithms

Index

Bibliography

[Ble26] Blender Foundation. Solidify modifier. Blender 5.2 LTS Manual, 2026. Accessed: 2026-08-27.

[edM] Morphology - Distance Transform — homepages.inf.ed.ac.uk. https://homepages.inf.ed.ac.uk/rbf/{H}{I}{P}{R}2/distance.htm. [Accessed 26-08-2026].

[FBKC+12] Andriy Fedorov, Reinhard Beichel, Jayashree Kalpathy-Cramer, Julien Finet, Jean-Christophe Fillion-Robin, Sonia Pujol, Christian Bauer, Dominique Jennings, Fiona Fennessy, Milan Sonka, et al. 3d slicer as an image computing platform for the quantitative imaging network. *Magnetic resonance imaging*, 30(9):1323–1341, 2012.

[HMVDW+20] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *nature*, 585(7825):357–362, 2020.

[MCZS19] Hugo I Medellin-Castillo and Jorge Zaragoza-Siqueiros. Design and manufacturing strategies for fused deposition modelling in additive manufacturing: a review. *Chinese Journal of Mechanical Engineering*, 32(1):1–16, 2019.

[MKM+23] Mahmoud Moradi, Mojtaba Karamimoghadam, Saleh Meiabadi, Shafqat Rasool, Giuseppe Casalino, Mahmoud Shamsborhan, Pranav Kattungal Sebastian, Arun Poulose, Abijith Shaiju, and Mohammad Rezayat. Optimizing layer thickness and width for fused filament fabrication of polyvinyl alcohol in three-dimensional printing and support structures. *Machines*, 11(8):844, 2023.

[reca] Filaflex 95A TPU Filament is compatible with all 3d printers — recreus.com. https://recreus.com/en/products/filaflex-95a?variant=45921633829123. [Accessed 27-08-2026].

[recb] TPU Filament Filaflex 60A, flexible filament for 3d printing — recreus.com. https://recreus.com/en/products/filaflex-60a?variant=45921624555779. [Accessed 27-08-2026].

[recc] TPU Filament Filaflex 82A, Flexible Filament for 3D Printing — recreus.com. https://recreus.com/en/products/filaflex-82a?variant=45921644773635. [Accessed 27-08-2026].

[She68] Donald Shepard. A two-dimensional interpolation function for irregularly-spaced data. In *Proceedings of the 1968 23rd ACM National Conference*, pages 517–524. Association for Computing Machinery, 1968.